

Wichtige Bibliotheken in Java

Java Standard Library, Collections Framework und Datenverarbeitung

java.lang

Grundlegende Klassen wie String, Math, Integer

java.util

Collections Framework und Hilfsklassen

java.time

Moderne Datums- und Zeitverarbeitung

java.math

Präzise mathematische Berechnungen

Warum Bibliotheken in Java wichtig sind

Das Problem

Grundlegende Funktionen jedes Mal selbst neu zu schreiben wäre unpraktisch. Typische Aufgaben wie Texte verarbeiten, Zahlen berechnen, Datumswerte verwalten oder Objekte speichern treten immer wieder auf.

Die Lösung: Java Standard Library

Java stellt eine große Sammlung fertiger, bewährter und getesteter Klassen bereit. Dadurch wird Code:

- kürzer und verständlicher
- zuverlässiger und wartbarer

Ein wichtiges Ziel ist daher, passende Bibliotheken für ein Problem zu kennen und einzusetzen.

Die Java Standard Library

Die Java Standard Library ist die mit Java mitgelieferte Klassenbibliothek. Sie umfasst viele **Pakete** – thematische Gruppierungen von Klassen, Interfaces und Hilfsmitteln.

1

java.lang

Grundfunktionen der Sprache

2

java.util

Verwaltung von Datenstrukturen

3

java.time

Datums- und Zeitverarbeitung

4

java.math

Präzise mathematische Berechnungen

Die Standardbibliothek ist das zentrale Fundament fast jedes Java-Programms.

Das Paket java.lang

Das Paket `java.lang` enthält die grundlegendsten Klassen von Java und wird **automatisch importiert** – kein explizites `import` nötig.

String

Texte darstellen und verarbeiten: `length()`, `toUpperCase()`, `substring()`, `equals()`

Math

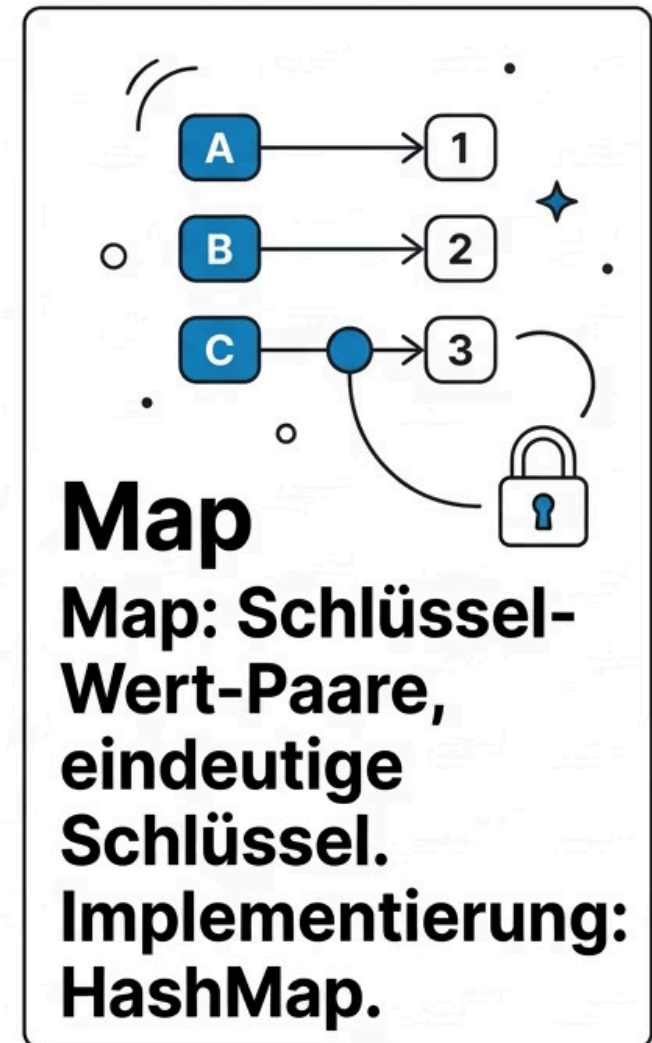
Statische Hilfsmethoden: `Math.round()`, `Math.sqrt()`, `Math.pow()`, `Math.max()`

Wrapper-Klassen

Integer, Double, Character, Boolean – kapseln primitive Typen als Objekte.
Beispiel: `Integer.parseInt("25")`

Das Java Collections Framework (JCF)

Das JCF ist ein standardisiertes System von Schnittstellen und Klassen zur Verwaltung von Objektmengen. In realen Anwendungen arbeitet man selten mit nur einem Objekt – es gibt viele Produkte, Benutzende, Bestellungen oder Termine, die gespeichert, gefunden, sortiert und gruppiert werden müssen.



❏ Grundprinzip: **Programmiere gegen Abstraktionen**, nicht nur gegen konkrete Implementierungen.

List – Geordnete Sammlungen

Eigenschaften einer List

- Reihenfolge bleibt erhalten
- Elemente über Index ansprechbar
- Doppelte Elemente erlaubt

Typische Verwendung

- Produkte in Eingabereihenfolge speichern
- Namen verwalten
- Objekte nacheinander durchlaufen

ArrayList

Die häufigste Implementierung. Geeignet wenn Elemente oft gelesen werden, am Ende neue Elemente hinzugefügt werden und eine flexible, wachsende Sammlung benötigt wird.

```
List<Produkt> produkte = new ArrayList<>();
```

Set – Mengen ohne Duplikate

HashSet

Wird verwendet, wenn Werte nur einmal vorkommen sollen:

- Kategorienamen
- Benutzernamen
- IDs und eindeutige Merkmale

```
Set<String> kategorien = new HashSet<>();
```

Eigenschaften eines Set

- Keine doppelten Elemente
- Keine indexbasierte Speicherung
- Geeignet für eindeutige Werte

Wenn mehrere Produkte derselben Kategorie vorhanden sind, speichert das Set den Kategorienamen trotzdem nur einmal. Das macht Set besonders nützlich zur **Beseitigung von Duplikaten**.

Map – Zuordnungen von Schlüssel zu Wert

Eigenschaften einer Map

- Jedem Schlüssel wird ein Wert zugeordnet
- Schlüssel sind eindeutig
- Werte über den Schlüssel auffindbar

Typische Zuordnungen

- Produktname → Produkt
- Kategorie → Liste von Produkten
- ID → Objekt

HashMap

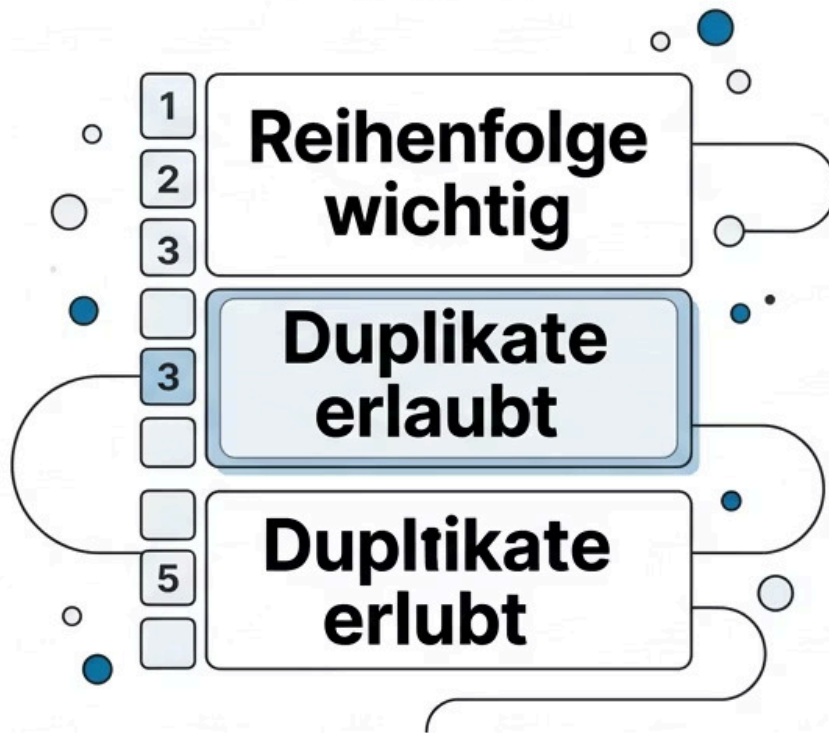
Die häufigste Implementierung für schnelle Zuordnungen.

```
Map<String, Produkt> produktMap = new HashMap<>();
```

Besonders wichtig, wenn Daten nicht nur gespeichert, sondern **gezielt zugeordnet und gefunden** werden sollen.

List, Set und Map im Vergleich

LIST



**Elemente nacheinander
verwalten**

SET



MAP



Paket java.time

Das Paket `java.time` dient zur modernen Datums- und Zeitverarbeitung. Frühere Datumsbibliotheken galten als unübersichtlich und fehleranfällig – `java.time` bietet eine klarere Lösung.

LocalDate

Repräsentiert ein Datum ohne Uhrzeit – für Geburts-, Liefer- oder Haltbarkeitsdaten.

`LocalDate.of(2026, 5, 10)`

Period

Berechnet den Abstand zwischen zwei Datumswerten in Jahren, Monaten oder Tagen.

`Period.between(heute, ablaufdatum)`

DateTimeFormatter

Formatiert Datumswerte für lesbare Ausgaben.

`DateTimeFormatter.ofPattern("dd.MM.yyy
y")` → 10.05.2026

Das Paket java.math – BigDecimal

Warum nicht double?

Der Typ `double` arbeitet mit binärer Gleitkommadarstellung. Dadurch entstehen **Rundungsfehler** – für Geldbeträge in kaufmännischen Programmen unerwünscht.

BigDecimal

Speichert Dezimalzahlen deutlich präziser. Geeignet für Preise, Rechnungsbeträge, Rabatte, Steuern und Summen.

Rechenoperationen erfolgen über Methoden:

- `add(...)`
- `subtract(...)`
- `multiply(...)`
- `divide(...)`

```
summe = summe.add(preis);
```

Bibliotheken im Zusammenspiel

Datenverarbeitung bedeutet mehr als nur Speichern – Daten müssen eingelesen, geprüft, sortiert, gefiltert, gruppiert und ausgegeben werden. Dabei arbeiten mehrere Bibliotheken zusammen.



Produkt-Objekt

`String` für den Namen, `BigDecimal` für den Preis, `LocalDate` für das Haltbarkeitsdatum



Produktverwaltung

`List` speichert mehrere Produkte geordnet, `Set` sammelt Kategorien ohne Duplikate



Datenanalyse

`Map<String, Produkt>` macht Produkte über Namen auffindbar, `Period` analysiert Datumsabstände

❏ Gute Datenverarbeitung entsteht durch das **Zusammenspiel mehrerer Bibliotheken**, nicht durch eine einzige Klasse.

Sortieren und Vergleichen mit Comparator

Sortierbedarf in Programmen

Objekte sollen oft nach bestimmten Kriterien geordnet werden:

- Alphabetisch nach Name
- Numerisch nach Preis
- Zeitlich nach Datum

Java stellt dafür fertige Werkzeuge bereit: `Collections.sort()`, `List.sort()` und `Comparator`.

Comparator

Ein `Comparator` beschreibt, nach welchem Kriterium Objekte verglichen werden – z. B. Produkte nach Preis sortieren.

Die Vergleichslogik wird vom Datenobjekt **getrennt**. Das macht Programme flexibler und besser erweiterbar, ohne Sortierlogik selbst programmieren zu müssen.

Bedeutung für objektorientierte Programme

Die Arbeit mit Bibliotheken ist eng mit objektorientierter Programmierung verbunden. Es entsteht eine klare Trennung:

Eigene Klassen

Modellieren den **fachlichen Inhalt** – z. B. die Klasse `Produkt` mit Name, Preis und Datum

Standardbibliotheken

Liefern **bewährte technische Werkzeuge** – String, BigDecimal, LocalDate, List, Set, Map

Das ist ein Zeichen guter Softwareentwicklung.

Fachliche Einordnung der Beispielklassen

1

Produkt

Verbindet `String`, `BigDecimal` und `LocalDate` direkt in einem Fachobjekt

2

ProduktVerwaltung

Nutzt eine `List` zur Verwaltung mehrerer Objekte – typischer JCF-Einsatz

3

DatenAnalyse

`Set` für eindeutige Kategorien, `Map` für Zuordnungen, `Period` für Datumsanalysen

4

Main

Verbindet alles und demonstriert die praktische Nutzung aller Bibliotheken in einem vollständigen Programm

Warum die Wahl der Bibliothek wichtig ist

Die Wahl der passenden Bibliothek ist Teil guter Programmgestaltung. Sie beeinflusst Lesbarkeit, Genauigkeit, Verständlichkeit, Wartbarkeit und Erweiterbarkeit.

→ Für Preise

`BigDecimal` ist geeigneter als `double`

→ Für geordnete Produktsammlungen

`List` ist passender als `Set`

→ Für eindeutige Kategorien

`Set` ist besser als `List`

→ Für gezielte Zuordnungen

`Map` ist sinnvoller als eine einfache Liste

❏ Gute Entwicklerinnen und Entwickler schreiben nicht alles selbst, sondern **wählen passende Werkzeuge bewusst aus.**

Merksätze auf einen Blick

Java Standard Library

Liefert fertige Klassen und Schnittstellen für typische Programmieraufgaben.

List · Set · Map

Geordnet mit Duplikaten · Eindeutig ohne Duplikate · Schlüssel-Wert-Zuordnung

java.lang / java.util / java.time / java.math

Grundfunktionen · Collections Framework ·
Datums-/Zeitverarbeitung · Präzise Berechnungen

BigDecimal · LocalDate · Comparator

Exakte Geldberechnungen · Datumsangaben ohne Uhrzeit ·
Flexibles Sortieren von Objekten

Zusammenfassung

Die Arbeit mit wichtigen Bibliotheken in Java ist ein **Grundbaustein professioneller Programmierung**.

Standard Library

java.lang, java.util, java.time und
java.math decken zentrale Anforderungen
ab

Collections Framework

List, Set und Map bieten unterschiedliche
Strukturen für unterschiedliche Probleme

Gutes Programmieren

Besteht nicht darin, alles selbst zu bauen,
sondern Bibliotheken fachlich sinnvoll
einzusetzen

Dadurch entstehen Programme, die **klarer, robuster und leichter erweiterbar** sind.